

Voice Controlled Authentication and Authorization System Report
By: Isra Ali

1. IMPLEMENTATION

Passphrase-based Authentication and Authorization

The passphrase-based system uses voice recognition to authenticate users. The system listens to specific phrases spoken by the user, then will grant access according to the recognized phrase and based on the user's role.

The implementation of the passphrase-based system is done using **Python3** programming language on **Linux Ubuntu** operating system, and code is run in the Ubuntu terminal using the command line interface (CLI). The main library that is used for voice recognition is **SpeechRecognition**, and it interfaces with Google Speech Recognition to process the audio inputs.

Tools and Technologies Used

- Python: The programming language used for writing the authentication and authorization code.
- SpeechRecognition: A python library that performs speech recognition by allowing the system to convert the speech to text [5].
- os Module: Fetch the current system username to assign it to the appropriate group.
- subprocess Module: execute system commands to update user group memberships.
- Microphone: captures the user's voice.
- Google Speech Recognition API: Used by the SpeechRecognition to convert the audio captured by the microphone into text format.

Documentation

I. Authentication

The program prompts the user to say one of the two predefined passphrases using the microphone: "hi system" for full administrative access, and "open system" for limited user access. Then, the user audio will be converted into text and compared with the authorized passphrases [5]. Any other input will result in authentication failure and access denial.

II. Authorization

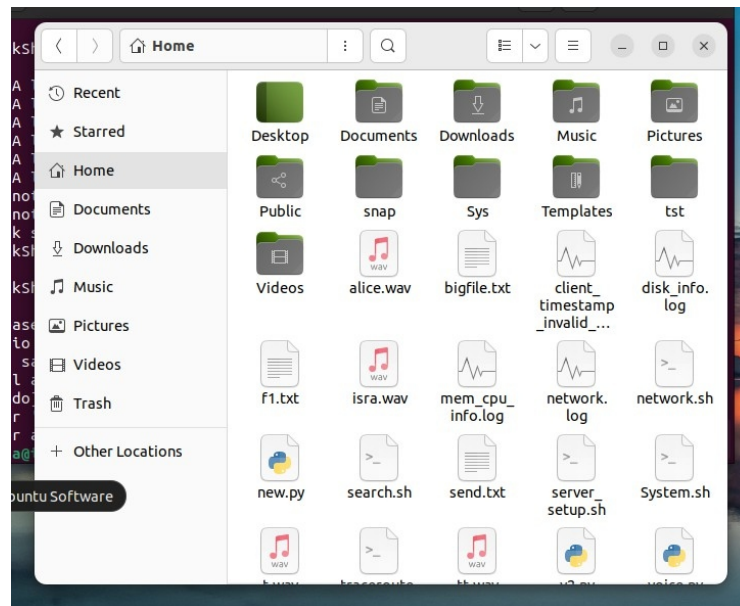
Once the user is authenticated, the program uses the subprocess and os modules to add the user to the appropriate system group. Users authenticated with "hi system" are added to the admin_group and will have full access with all permissions to files under /home/client1 directory. Also, users authenticated with "open system" are added to the user_group and will be granted access only to /home/client1/Users directory and will have read-write-execute permissions under the same directory.

The images below illustrate both the authentication and authorization processes:

Authentication using "**hi system**":

```
K
Please say the passphrase to authenticate:
Audio captured successfully!
You said: hi system
Full access granted.
[sudo] password for isra:
User 'isra' added to the group 'admin_group'.
User added to 'admin_group'. Full access granted.
isra@isra-VirtualBox:~$
```

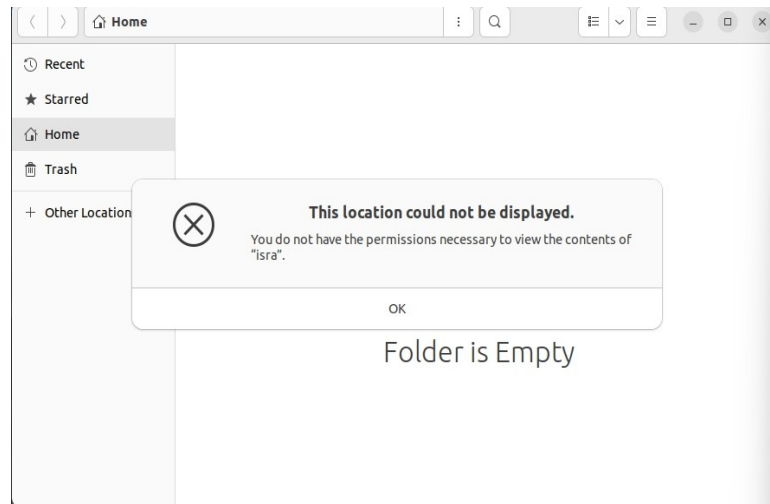
User granted **full access**:



Authentication using “**open system**”:

```
Please say the passphrase to authenticate:
Audio captured successfully!
You said: open system
Limited access granted.
User 'client1' added to the group 'user_group'.
User added to 'user_group'. Limited access granted.
client1@client1-VirtualBox:~/Users$
```

User “client1” granted **limited access**:



Using non-authenticated passphrase

```
isra@isra-VirtualBox:~$ python3 script.py
Please say the passphrase to authenticate:
Audio captured successfully!
You said: 123
Authentication failed! Incorrect passphrase.
Access denied.
isra@isra-VirtualBox:~$
```

2. Appendix:

➤ Code for the passphrase-based system:

```
import speech_recognition as sr
import os
import subprocess

# Initialize recognizer
recognizer = sr.Recognizer()

# The correct passphrases for authentication
AUTHORIZED_USER_PASSPHRASE = "open system"
AUTHORIZED_ADMIN_PASSPHRASE = "hi system"

def add_user_to_group(username, group_name):
    try:
        # Use subprocess to add user to the specified group
```

```

        subprocess.run(["sudo", "usermod", "-aG", group_name, username],
check=True)
        print(f"User '{username}' added to the group '{group_name}'.")
    except subprocess.CalledProcessError:
        print(f"Failed to add user '{username}' to the group '{group_name}'.")

def authenticate_voice():
    with sr.Microphone() as source:
        print("Please say the passphrase to authenticate:")

        # Adjust for ambient noise before listening
        recognizer.adjust_for_ambient_noise(source)

    try:
        # Capture the audio from the microphone
        audio = recognizer.listen(source, timeout=5) # timeout :5 sec
        print("Audio captured successfully!")

    except sr.WaitTimeoutError:
        print("No speech detected within the timeout period.")
        return None
    except sr.RequestError:
        print("Could not connect to the Google Speech Recognition service.")
        return None
    except sr.UnknownValueError:
        print("Speech was not recognized.")
        return None

    # Try recognizing the speech and check for the passphrase
    try:
        recognized_text = recognizer.recognize_google(audio).lower()
        print("You said: " + recognized_text)

        if recognized_text == AUTHORIZED_USER_PASSPHRASE:
            print("Limited access granted.")
            return "user"
        elif recognized_text == AUTHORIZED_ADMIN_PASSPHRASE:
            print("Full access granted.")
            return "admin"
        else:
            print("Authentication failed! Incorrect passphrase.")
            return None

    except sr.UnknownValueError:
        print("Could not understand the audio.")
        return None
    except sr.RequestError:
        print("Could not request results from the Google Speech Recognition
service.")
        return None

```

```

# Authentication step
user_role = authenticate_voice()
if user_role == "user":
    # Authorization step for limited access user
    username = os.getlogin() # Get the current system username
    add_user_to_group(username, "user_group")
    print("User added to 'user_group'. Limited access granted.")

elif user_role == "admin":
    # Authorization step for admin
    username = os.getlogin() # Get the current system username
    add_user_to_group(username, "admin_group")
    print("User added to 'admin_group'. Full access granted.")

else:
    print("Access denied.")

```

➤ **Passphrase-based system manual:**

Initial requirements:

1. Download VirtualBox and Ubuntu ISO file to set up the virtual machine.
2. Install Ubuntu inside VirtualBox then create an account on your Linux Ubuntu virtual machine.
3. Make sure that you have python3.x installed on your system.
4. Install SpeechReconition and puaudio libraries using the command:

pip install SpeechReconition pyaudio.

5. To use Google Speech Recognition API, ensure that your system is connected to the internet.
6. Form the settings > Audio, enable audio input to allow the microphone to capture the voice.
7. The user running the script must have sudo privileges since the code uses system-level commands.
8. Create two groups one for the administrators and the other for the regular users using the following commands:

**sudo groupadd
admin_group sudo
groupadd user_group**

9. Change the permission for admin_group to allow admins have full access to home directory:

**sudo chown -R :admin_group /home/client1
sudo chmod -R 770 /home/client1**

10. Change the permission for user_group to prevent users from having access to home directory and only have access to /home/client1/Sys directory:

**sudo chown -R :user_group /home/client1/Users
sudo chmod -R 770 /home/client1/Users
sudo chmod -R 700 /home/client1
sudo chmod -R 700 /home/**

Running the Code

1. To run the code, either copy the given code into a file with .py extension and run it using the command: **python3 “name of your file”.py**, or download the file “voice_authentication.py” inside this repo. Then run it using the same command: **python3 voice_authentication.py**.
2. The program will prompt the user to say the passphrase, the user must be close to the microphone while speaking for better passphrase recognition.
3. The script checks if the recognized speech matches one of the passphrases and grant access based on that.